

Supplemental Material for “Details Where They Matter: Understanding Projection Spaces Using In-Place Summary Visualizations”

Christian A. Steinparz , Yannick Metz , Andreas Hinterreiter , Manuela Waldner ,
Mennatallah El-Assady , and Marc Streit 

This document provides the implementation details referenced by the paper’s Method and Implementation sections, covering the propagation, semantic-zoom, and inset-placement components. The appendix letters are labels for this document only and do not correspond to sections of the paper. File paths refer to the public repository <https://github.com/jku-vds-lab/tvcg26-details-on-demand> at commit 3fba4f6, relative to `packages/app/` unless stated otherwise.

APPENDIX A

PROPAGATING INTEREST THROUGH A FULLY CONNECTED SPACE

Interest propagation spreads the degree of interest from the seeds, the states a user selected or matched with a query, to the rest of the projection. Besides following trajectories, interest travels through the projected space itself, so states near a seed gain interest even without a trajectory connection. This appendix describes how that spatial channel stays interactive, by working on a distance field over a grid instead of comparing every pair of states.

A.1 Rasterization grid and sampling

The spatial channel needs the distance from every state to the nearest seed, and computing that directly means comparing every state against every seed. We therefore measure distances on a rasterization grid, a raster of square cells over the projection, rather than between individual states. We derive the grid from the data extent alone, with 1024 square cells along the longer axis and proportionally fewer along the shorter one, so the same field results regardless of the current viewport or zoom level. Selected states are rasterized into a seed mask, and every state reads its distance back by bilinear sampling of the resulting field. The grid construction and the sampling live in `src/doiPropagation/fieldDistanceCore.ts`.

Christian A. Steinparz, Andreas Hinterreiter, and Marc Streit are with Johannes Kepler University Linz, 4040 Linz, Austria (e-mail: christian.steinparz@jku.at, andreas.hinterreiter@jku.at, marc.streit@jku.at).

Yannick Metz and Mennatallah El-Assady are with ETH Zürich, 8092 Zürich, Switzerland (e-mail: yannick.metz@inf.ethz.ch, melassady@ai.ethz.ch).

Manuela Waldner is with TU Wien, 1040 Wien, Austria (e-mail: manuela.waldner@tuwien.ac.at).

Manuscript submitted September 16, 2026.

A.2 Exact distance transform in the first round

Once the seeds are rasterized, every grid cell needs its straight-line distance to the nearest seed cell. A distance transform computes that for the whole grid at once, and the first spatial round runs an exact Euclidean one on the seed mask in time linear in the number of cells. We use a port of the Felzenszwalb–Huttenlocher algorithm [1] (`edtSquared` in `fieldDistanceCore.ts`), which sweeps each row and then fits a lower envelope of parabolas along each column. The test suite checks that this port matches SciPy’s `distance_transform_edt` exactly. Rasterization, exact transform, and bilinear read-back together form round zero of every propagation.

A.3 Chamfer sweep in later rounds

After the first round, chain-raised states re-enter the spatial channel as weakened sources (A.4), so later rounds recompute the field from many sources with their own distance offsets. A chamfer sweep is a distance transform that passes over the grid twice and lets every cell take the smallest value among its neighbors plus the step to them. We use one (`respreadDistanceSweep` in `fieldDistanceCore.ts`) that folds the eight neighbors of each cell with the true straight and diagonal cell distances. Sources that share a cell enter at the minimum of their offsets, never additively. Because the grid has no barriers, the sweep is exact for the chamfer metric, which closely approximates Euclidean distance at the same linear cost.

A.4 Falloff family and exact propagation formulas

In the spatial channel, the interest a state receives from the nearest seed decreases with its distance to that seed, and the falloff f_{wprox} of the paper’s proximity channel is the curve that describes this decrease. The shape of that curve is what a user feels when dragging the proximity slider, so the interface offers three curves next to the slider, logarithmic, plateau, and linear (Fig. A.1), with the logarithmic one as the default. It holds interest high near a seed and then fades to zero within a bounded radius, so low slider settings keep interest local instead of giving every distant state a faint value. The proximity weight sets the scale over which every curve decays.

Three quantities define the curves. The normalized distance $u = d_{emb}(x, y)/D$ is the projected distance between two states

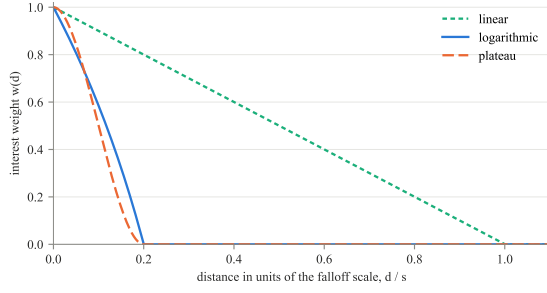


Fig. A.1. Reach of the three falloff curves at the same slider position, plotted against distance in units of the falloff scale, d/s with $d = d_{\text{emb}}/D$ and $s = w_{\text{prox}}/(1 - w_{\text{prox}})$, here for $w_{\text{prox}} = 0.1$. The linear curve spreads interest over the full scale s , while the logarithmic default and the plateau curve stop at $s/5$, so the same slider setting gives a much more local effect. The logarithmic curve keeps interest high near the seed and then drops quickly, the plateau curve stays flat even longer before its cliff.

divided by the extent D of the projection along its longer axis, so $u = 1$ spans the whole projection. The falloff scale $s = w_{\text{prox}}/(1 - w_{\text{prox}})$ turns the slider value into the distance over which a curve decays, growing without bound as the slider approaches 1. The support radius $r = s/5$ is the distance at which the logarithmic and plateau curves reach zero. The curves are

$$f^{\log}(u) = \begin{cases} \log_2(2 - u/r) & u < r \\ 0 & u \geq r, \end{cases}$$

$$f^{\text{plateau}}(u) = t^2(3 - 2t), \quad t = \min(1, \max(0, 1 - u/r)),$$

$$f^{\text{lin}}(u) = \max(0, 1 - u/s).$$

Setting $w_{\text{prox}} = 0$ switches the channel off, and $w_{\text{prox}} = 1$ removes the decay, so every state receives the full value.

A state that gained its interest through a trajectory chain acts as a weakened seed in the spatial channel. The implementation continues the falloff from where that value sits instead of multiplying by it: the state enters at the distance offset $f^{-1}(\text{DoI}(y))$ at which the falloff would have produced its value, and a state x receives $f(f^{-1}(\text{DoI}(y)) + u)$. This differs from the product form $\text{DoI}(y) \cdot f(u)$ of the paper's figure in that a chain of weakened seeds never reaches farther than the radius of the original seed. Because f is monotone, the maximum over sources in the paper's model becomes a minimum over effective distances, which is what the distance field computes. The inverses have closed forms, for example $f_{\log}^{-1}(v) = r(2 - 2^v)$, and both directions live in `src/doiPropagation/falloff.ts`.

A.5 Convergence bound

Interest that arrives through a trajectory can seed further spatial spread, and spatially gained interest can feed further trajectory hops, so the channels run in turn until nothing changes. The convergence bound guarantees that this alternation stops after a few rounds. The alternation ends once neither channel raises any state by more than a small epsilon (`runConvergedAlternationCore` in `convergedField.ts`). Both channels only raise values and every hop attenuates by a factor below 1, so contributions

shrink geometrically. We cap the alternation at 12 rounds, and every real-data case we have observed converged within 9.

A.6 GPU motion lane and precision deltas

While a user drags a propagation slider, the field is recomputed for every pointer position, and the exact CPU alternation of A.5 is too slow for that. The GPU motion lane is a faster path in which the same alternation runs in single precision on the GPU with a fixed round count, so the preview keeps up with the pointer (`src/gl/shaders/convergedMotion.ts`, `src/gl/core/systems/ConvergedMotionSystem.ts`). On slow hardware the lane steps down to a coarser grid of 512 and then 256 cells for the rest of the session (`src/doiPropagation/gpuMotionAdaptiveRes.ts`). The precision deltas, the preview's deviations from the CPU field, are negligible at full resolution and a few hundredths at the coarsest tier, so the preview is always replaced by the exact field (A.7).

A.7 Preview scheduling and commit

The GPU preview of A.6 is approximate and must never become the stored value, yet the user should not wait for the exact field either. A commit replaces the preview by the exact field. While the GPU lane renders a preview, a background worker computes the exact CPU field for the same slider values and fades it in behind, so persisted values always come from the full-precision engine (`src/hooks/useDoIPropagation.ts`). A resting pointer commits automatically after a short settle time. For CPU-lane drags, the chain contribution is frozen at drag start, so the preview is a lower bound of the exact field and equals it at the freeze point (`fieldPreviewCore.ts`). Stale results are discarded by epoch counters.

APPENDIX B

SEMANTIC-ZOOM BOOKKEEPING

The states that carry enough interest are grouped into a cluster hierarchy, and the semantic zoom decides at which level of that hierarchy the projection is annotated. Zooming in, panning, or changing interest moves this cut, so clusters split or merge and their insets appear, disappear, or change members. The visible result should be stable and immediate, which means the same cluster has to be recognized across rebuilds, ancestry between candidate clusters has to be answered quickly, unchanged cuts should not be recomputed, insets should return to where they were, and decisions near a threshold should not flicker. This appendix describes the bookkeeping that provides those five properties.

B.1 XOR leaf-signature cross-referencing of the two HDBSCAN tree representations

HDBSCAN [2] first builds a single-linkage hierarchy over all points and then condenses it into the tree of clusters that persist over a range of density levels, and it computes its stability score only for those condensed clusters. The

hdbscan Python library we use in the preprocessing exposes these as two separately numbered structures, the full single-linkage tree and the condensed tree, without a mapping between them. The semantic zoom cuts the full tree, so every node of it that corresponds to a condensed cluster has to receive that cluster’s stability, and the only property the two structures share is which points belong to a node. Listing the member set of every node and comparing the lists costs up to $O(n^2)$ time and memory for n points, because a node can hold up to n points and there are $n-1$ internal nodes. The preprocessing (`scripts/data-prep/preprocess_dataset_generate_knng.py` at the repository root) instead computes a 64-bit signature per node in one bottom-up pass over each tree, where each leaf contributes a hash of its index and each internal node the XOR of its children’s signatures, and joins the trees through a hash map on these signatures in time linear in the number of nodes. Because XOR is commutative and associative, the same member set yields the same signature regardless of tree shape.

B.2 Euler-tour ancestry timestamps

A zoom cut is a set of clusters that do not nest, so that no shown cluster contains another shown cluster. We build it greedily, taking candidates in order of decreasing stability and accepting each one unless it is an ancestor or a descendant of a cluster already accepted (`getTopNClusters` in `src/clustering/clusteringService.ts`). This asks, for every candidate against every accepted cluster, whether one is an ancestor of the other. Parent and child pointers exist in the tree, but they only reach the immediate neighbor, so answering the question through pointers means walking up towards the root, and a single-linkage dendrogram over n points can be almost n levels deep. We therefore assign every node an entry and an exit timestamp from one depth-first traversal after each hierarchy rebuild, the Euler tour of the tree. A node is a descendant of another exactly when its timestamp interval lies inside the other’s, so each ancestry question costs two integer comparisons.

B.3 Zoom-cut result cache

The zoom cut is requested on every pan, zoom, slider change, and re-render, but computing it means walking the cluster hierarchy, and most requests arrive with inputs identical to the previous one. The semantic-zoom service (`src/semanticZoom/semanticZoomService.ts`) therefore remembers the last cut together with a signature of everything the cut depends on, the viewbox rounded to a coarse grid, the canvas size, the entity budget, the split, ranking, and hysteresis parameters, whether a selection exists, and a revision counter of the hierarchy. A request whose signature equals the stored one returns the stored cut without any computation. The cache needs no explicit invalidation, because every recluster or reload increments the revision counter and thus changes the signature.

B.4 Inset position memory

When the cut changes, insets disappear and later come back, for example when a user zooms in and out again. Placing

a returning inset from scratch would let the annealer put it somewhere new each time, which breaks the mental map, so we remember where every inset was and put it back there. The memory (`src/hooks/reconcileClusterItems.ts`) is keyed twice, once by the element id and once by an order-independent digest of the cluster’s members, so a cluster that returns with the same members is recognized even when a rebuild gave it a new id. A restored inset re-enters cold, so the annealer only moves it if something now conflicts with it. An inset without a remembered position starts just outside its cluster’s contour, and the memory is cleared when the dataset changes.

B.5 Hysteresis constants

Several decisions in the pipeline compare a continuous value against a threshold, and a value that hovers around its threshold would make the decision flip back and forth on every small change. A hysteresis band prevents this by demanding a clearly stronger signal to enter a state than to stay in it. Three such bands keep the zoom cut and the inset layout from flickering.

- *Cluster activation.* `HysteresisManager` (`src/semanticZoom/hysteresis.ts`) compares each cluster’s saliency to that of the K -th ranked candidate, the border of the cut. A cluster that is already shown stays in the cut as long as its saliency remains above 85% of that border, whereas a cluster that is not yet shown enters only once its saliency exceeds the border by 10% (defaults in `src/store.ts`).
- *Viewport clamp.* Insets that drift off screen are clamped back into view, and the clamp and the annealer’s move generator both use the same margin of 6% of the current viewbox, so an inset near the edge is not pushed back and forth between the two (`src/annealing/InsetOptimization.ts`, `src/hooks/useLayoutEngine.ts`).
- *Reheat dead-band.* The cost function penalizes any inset overlap, but selective reheating fires only once the overlap reaches 8% of the smaller box (`useLayoutEngine.ts`), which separates “penalized” from “worth re-warming”.

APPENDIX C INSET PLACEMENT

C.1 Objective and cost terms

Inset placement has to keep each inset close to its own cluster while never letting it cover another inset, cross a leader line, or sit inside a foreign contour. Each of these requirements is expressed as a cost term, and their sum is the placement objective that the annealer minimizes (`computeVisualElementsCost` in `src/annealing/InsetOptimization.ts`). Default weights come from `src/store.ts` and can be changed at runtime in the expert settings panel. Table C.1 lists the terms that shape the default layout. In the default configuration one soft term, the distance to the target ring, is subordinate to three hard penalties, and

TABLE C.1

COST TERMS OF THE INSET-PLACEMENT OBJECTIVE WITH THEIR DEFAULT WEIGHTS. ALL TERMS ARE SUMMED INTO ONE SCALAR BY `COMPUTEVISUALELEMENTSCOST` IN `SRC/ANNEALING/INSETOPTIMIZATION.TS`, AND THE DEFAULTS ARE THE CORRESPONDING KEYS IN `SRC/STORE.TS`. A FEW FURTHER WEIGHTS EXIST IN THE CODE AS VARIANTS OF THESE TERMS, SUCH AS A SOFT OVERLAP BARRIER OR A SOFT LEADER-LINE CROSSING COUNT. THEY DEFAULT TO ZERO OR ARE DOMINATED BY THE HARD PENALTIES, AND ARE OMITTED.

Term	Symbol	What it penalizes	Default
Distance to target ring (soft)	w_{dist}	deviation of the inset from the target radius around its own cluster contour	5.0
Mental-map stability (soft)	w_{map}	movement away from the position of the previous layout	0
Own-source overlap (soft)	w_{src}	covering the inset's own source box	0
Data occlusion (soft)	w_{occ}	data points and edges under the inset box	0
Overlap avoidance (hard)	p_{ov}	any overlap between insets, difference insets, or user-pinned insets, a fixed penalty plus a barrier growing with the overlapped fraction	5000
Leader-line crossing (hard)	p_{cross}	each crossing pair of leader lines	5000
Foreign-contour intrusion (hard)	p_{intr}	an inset inside another cluster's contour, doubled when its center lies inside	5000

the remaining soft terms are switched off. For a layout P that assigns every inset i a position, the objective is

$$\begin{aligned}
 E(P) = & \sum_i (w_{\text{dist}} d_i + w_{\text{map}} m_i + w_{\text{src}} o_i + w_{\text{occ}} q_i) \\
 & + p_{\text{ov}} \sum_{i < j} [a_{ij} > 0] (1 - \ln(1 - a_{ij})) \\
 & + p_{\text{cross}} c + p_{\text{intr}} \sum_i k_i.
 \end{aligned}$$

Here r_i is half the diagonal of inset i , which normalizes all distances. The distance term $d_i = |\rho_i - 0.5r_i|/r_i$ measures how far the inset's distance ρ_i to its own cluster contour deviates from the target ring at half its own size (the factor 0.5 is `contourTargetRadiusMultiplier`). The movement term m_i is the distance to the previous position divided by r_i , o_i is the fraction of the inset's own source box that it covers, and $q_i = \ln(1 + \text{density})$ counts data points and edges under the box per unit area. For a pair of insets, including difference insets, a_{ij} is the overlapped area divided by the smaller box, so the hard term adds a fixed penalty plus a barrier that grows without bound as one box disappears under the other. Finally, c is the number of crossing leader-line pairs, and k_i counts foreign contours that the inset's box intersects, with 2 instead of 1 when the inset center lies inside the contour.

The hard penalties are three orders of magnitude larger than the active soft weight, so any hard violation outweighs every soft consideration, and the ordering between hard and soft terms comes from magnitude alone. The overlap term has two parts for a reason. The step of 1 charges the full penalty as soon as two boxes touch at all, which is what makes overlap forbidden. A pure step would cost the same at 10% and at 90% overlap, so the annealer would not know which move helps. The barrier $-\ln(1 - a_{ij})$ starts at zero and grows without bound as one box disappears under the other, so every move that reduces overlap is downhill, even when the annealer is cold and accepts only improvements. The intrusion count k_i doubles for the same reason when an inset center lies inside a foreign contour. Floating difference insets, anchored at the midpoint between two parents, and user-pinned insets take part in the same overlap term, and a pinned inset is never moved itself, so the other inset moves away.

C.2 Temperature schedule and cooldown constants

Inset positions are refined by simulated annealing (simulatedAnnealing in `src/annealing/simulatedAnnealing.ts`), which proposes random moves whose size shrinks as a temperature cools, so insets explore first and settle later. The temperature schedule is the rule by which that temperature falls. Each inset carries its own temperature, which is lowered only when one of its own moves is accepted, so insets settle independently of one another. Every inset starts at temperature 1, cools by a factor of 0.92 per accepted move, and sleeps below 0.001, and a run ends early once every inset sleeps. Proposed moves are jittered in proportion to the current temperature, up to a jitter strength of 24 screen pixels at temperature 1, and divided by the zoom factor, so moves keep the same size on screen. A run is capped at 1200 annealing steps. The cooling rate, the jitter strength, and the step cap can be changed at runtime in the expert settings panel, while the sleep threshold is fixed.

C.3 Reheating

Once an inset has cooled, the annealer no longer moves it, but the layout around it keeps changing, so a settled position can turn bad. Reheating warms it up again so the annealer can move it. An inset is not re-warmed by every layout change but only when its position has become untenable, which means being pushed out of view, severe overlap with another inset, intrusion into a foreign contour, or a leader-line crossing (`src/hooks/useLayoutEngine.ts`). Reheats are rate-limited, and after a few attempts the engine gives up until the environment changes. Reheating and annealing pause during zoom gestures to preserve the mental map, and user-pinned placements are never reheated. New or membership-changed elements re-enter at full temperature.

C.4 Non-blocking refinement scheduling

The annealer keeps refining positions while the user interacts, and running it to completion in one go would freeze the interface. Non-blocking refinement cuts the work into slices that fit inside one frame. The annealer runs inside the animation-frame loop and yields after a frame budget of 6ms, the figure quoted in the paper, returning the best positions found so far when

the budget is exceeded (`src/layout/engineIdle.ts`, `simulatedAnnealing.ts`). The loop goes to sleep after a few consecutive converged frames, where frames with zooming, reheating, or store changes do not count as converged. A periodic watchdog compares a cheap input signature so that a missed wake-up cannot strand the layout.

C.5 Position memory across cluster reappearance

A cluster that leaves the cut and later returns should not have its inset placed from scratch, because the annealer might put it somewhere new. The membership-keyed position memory of [B.4](#) serves inset placement as well. When a cluster reappears with the same member set, its inset is restored at its previous world position with temperature 0, so reappearance does not restart the optimization.

REFERENCES

- [1] P. F. Felzenszwalb and D. P. Huttenlocher, “Distance transforms of sampled functions,” *Theory of Computing*, vol. 8, no. 1, pp. 415–428, 2012.
- [2] R. J. G. B. Campello, D. Moulavi, and J. Sander, “Density-based clustering based on hierarchical density estimates,” in *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, ser. Lecture Notes in Computer Science, vol. 7819. Springer, 2013, pp. 160–172.